

Report conversation

in unit tests, what comes first in different languages -
actual and expected?

In unit testing, whether you should write the expected value before the actual value often depends on the testing library and language you are using. Here's a breakdown of some of the popular testing libraries and languages and their conventions:

1. JUnit (Java):

java

 Copy code

```
assertEquals(expected, actual);
```

2. MSTest (C#):

csharp

 Copy code

```
Assert.AreEqual(expected, actual);
```

3. NUnit (C#):

csharp

 Copy code

```
Assert.AreEqual(expected, actual);
```

4. xUnit (C#):

It's a bit different in xUnit. Instead of an `AreEqual` method, it has an `Equal` method.

csharp

 Copy code

```
Assert.Equal(expected, actual);
```

5. Python's unittest :

python

 Copy code

```
self.assertEqual(expected, actual)
```

6. Go's testing package:

Go doesn't have an assertion library built into its standard library, but a common one used is `testify`.

```
go
```

[Copy code](#)

```
assert.Equal(t, expected, actual)
```

7. PHPUnit (PHP):

```
php
```

[Copy code](#)

```
$this->assertEquals(expected, actual);
```

8. RSpec (Ruby):

```
ruby
```

[Copy code](#)

```
expect(actual).to eq(expected)
```

Note that in RSpec, you start with the actual value and compare it to the expected.

9. Jest (JavaScript/TypeScript):

```
javascript
```

[Copy code](#)

```
expect(actual).toBe(expected);
```

Similar to RSpec, in Jest you also start with the actual value.

Remember, the primary goal of writing tests is to clearly convey the intent and ensure correctness. Whichever order you choose, it's essential to be consistent throughout your codebase and make sure that other developers can easily understand your tests.

what about OCaml and Elm?

Certainly, let's address both:

1. OCaml:

OCaml doesn't have a standardized unit testing library in its core, but there are

several libraries available for this purpose. One of the popular ones is `ounit`. In `ounit`, the typical order is:

```
ocaml
```

[Copy code](#)

```
assert_equal ~printer:[printer_function] expected actual
```

Here, `expected` comes before `actual`, and the `printer` argument can be used to specify how the value should be printed if the assertion fails.

2. Elm:

Elm's primary package for unit testing is `elm-test`. With `elm-test`, the convention is similar to some other languages where you start with the actual value and compare it to the expected. Here's how it typically looks:

```
elm
```

[Copy code](#)

```
Test.equal expected actual
```

So, in Elm's `elm-test`, the `expected` value comes first, followed by the `actual`.

In both OCaml and Elm, like other languages, the important part is to be consistent and clear in your testing practices, ensuring that tests are readable and maintainable.